

Halcon_IC

IC 芯片视觉检测系统

项目技术文档

v1.0 2026-09-04

基于 .NET 8 / Prism 8 / Halcon 24.11
GigE Vision 工业相机 + 雷赛运动控制卡

目 录

I 系统架构

项目定位、技术栈、分层架构、业务流程

II 核心模块设计

ViewModels、Services、数据访问、基础设施与模型

III 通信机制设计

事件驱动通信、消息总线、生命周期管理

IV 核心技术实现

视觉算法、多线程、非托管资源管理

V 已知风险与优化建议

内存泄漏、线程安全、硬件耦合、数据库风险

VI 文档修订历史

版本追踪

I. 系统架构

1. 项目定位

本项目为 IC 芯片（8 引脚 LM386 功放芯片）的视觉检测系统，基于 Halcon 24.11 形状模板匹配技术，对传送带上高速通过的芯片进行实时检测，判断针脚完整性并剔除不合格品。

- 检测对象：8 引脚 IC 芯片（两侧各 4 个针脚）
- 检测内容：针脚缺失检测，左右各 4 针脚完整性判定
- 动作流程：检测→判定→触发气缸剔除不合格品
- 设备编号："奶龙包"（配置文件中可配置）

2. 技术栈

层次	技术	版本/说明
运行时	.NET 8 WPF	Windows 桌面应用
MVVM 框架	Prism 8	Region/Module/EventAggregator/Dialog
视觉算法	Halcon 24.11	GigEV2 / DirectShow / 形状模板匹配
图表	LiveChartsCore (SkiaSharp)	统计曲线显示
数据库	SQL Server + EF Core 8	产品检测记录持久化
运动控制	雷赛 LTSMC 控制卡	传送带启停与调速
IO 控制	传感器 + 电磁阀	触发检测与不合格品剔除
配置管理	appsettings.json	强类型配置绑定

3. 分层架构

项目采用 7 层模块化架构，各层职责明确：

层级	项目	职责
1. 入口层	Halcon_IC	App.xaml 启动、Prism Shell、Region 定义
2. UI 层	Vision	View (XAML) / ViewModel / Service 实现
3. 核心业务	Core	接口定义、事件、模型、枚举
4. 硬件控制	Motion	运动控制卡、传感器、电磁阀
5. 数据访问	Data	EF Core DbContext、Repository、统计服务
6. 基础设施	Infrastruce	配置中心、日志服务、Halcon 运行时
7. 硬件驱动	External	LTSMC.dll (雷赛控制卡 SDK)

依赖方向：入口层 → UI 层 → 核心业务层，硬件控制/数据访问/基础设施层均依赖核心业务层，各层之间通过接口解耦，通过 Prism DI 容器注册服务。

4. 核心业务流程图

系统启动后，用户按以下 9 步流程完成检测任务：

- 第 1 步：打开相机 —— 根据 appsettings.json 配置连接 GigE 相机
- 第 2 步：开始采集 —— 启动后台采集线程，循环 GrabImage
- 第 3 步：创建模板 —— 在图像窗口框选芯片区域，创建形状模板
- 第 4 步：绘制检测区域 1 —— 绘制左侧针脚检测区域

- 第 5 步：绘制检测区域 2 —— 绘制右侧针脚检测区域
- 第 6 步：保存模板 —— 将模板坐标与检测区域配置序列化为 JSON
- 第 7 步：开始检测 —— 启动 BlockingCollection 生产者-消费者检测引擎
- 第 8 步：实时检测 —— 每帧经模板匹配 + 针脚计数 + 合格判定
- 第 9 步：停止检测/关闭相机 —— 释放非托管资源

5. Prism Region 导航

主界面通过 Prism Region 管理多视图切换，注册的导航视图包括：

视图	ViewModel	功能
VisionWindow	VisionWindowViewModel	相机控制、模板管理、检测主界面
StatisticView	StatisticViewModel	实时统计与历史曲线
SystemLogView	SystemLogViewModel	系统日志实时显示
DetectionRecordView	DetectionRecordViewModel	检测结果实时记录列表
MotionControlView	MotionControlViewModel	传送带控制（预留）

II. 核心模块设计

1. ViewModels 设计

1.1 VisionWindowViewModel（核心 ViewModel）

VisionWindowViewModel 是整个系统的中枢 ViewModel，编排相机采集 → 模板创建/加载 → 检测 → 结果发布的完整流程。构造函数注入 7 个服务：ICameraService、ITemplateService、IDetectionService、IEventAggregator、ILogService、IConfigService、IDialogService。

命令列表（共 12 个 DelegateCommand）：

命令	绑定属性	前置条件
OpenCameraCmd	!IsCameraOpen	——
CloseCameraCmd	IsCameraOpen	——
StartGrabCmd	IsCameraOpen && !IsDetecting	相机已打开
StopGrabCmd	IsCameraOpen && !IsDetecting	相机已打开
CreateTemplateCmd	IsCameraOpen && !IsDetecting	相机已打开
CreateCheckXld1Cmd	IsTemplateCreated	模板已创建
CreateCheckXld2Cmd	IsTemplateCreated	模板已创建
SaveTemplateCmd	IsTemplateCreated	模板已创建
LoadTemplateCmd	!IsDetecting	非检测中
LoadReferenceImageCmd	!IsDetecting	非检测中
StartDetectCmd	IsTemplateCreated && !IsDetecting	模板已创建
StopDetectCmd	IsDetecting	检测中

事件（View 订阅）：

- ImageGrabbed —— 采集图像就绪事件，View 用于显示实时图像
- DetectionResultUpdated —— 检测结果更新事件，View 用于绘制叠加图形
- RequestTemplateCreate —— 请求绘制模板区域
- RequestCheckXld1Create / RequestCheckXld2Create —— 请求绘制检测区域
- RequestDrawRoi / RequestClearRoi —— ROI 绘制与清除

1.2 StatisticViewModel

负责统计数据加载与 LiveCharts 曲线绑定。每 5 秒自动刷新实时计数（TotalCount/OkCount/NgCount/Yield），支持按 1 小时/4 小时/24 小时范围查询历史聚合曲线。

- 数据来源：IStatisticsService（端侧聚合，按分钟 GROUP BY）
- Y 轴双轴：左轴=总数（SteelBlue），右轴=良率%（Orange）
- 自动刷新：IPollTask 每 5 秒回调 OnAutoRefreshAsync
- 手动刷新：RefreshCmd + 时间范围切换触发

1.3 DetectionRecordViewModel

实时监听 DetectionResultEvent（UIThread），维护最多 200 条检测记录的 ObservableCollection，用于列表显示。

1.4 SystemLogViewModel

实时监听 LogPubSubEvent (UIThread)，维护最多 200 条日志记录的 ObservableCollection，支持一键清空。

2. Services 设计

2.1 CameraService (ICameraService 实现)

基于 Halcon 24.11 GigEV2 接口的相机服务。通过 HFramegrabber OpenFramegrabber 连接，后台 Task 循环 GrabImage，每帧 Clone 后通过 ImageGrabbedEvent 发布。

- 接口选择：GigEV2（默认）/ DirectShow（MediaFoundation，由配置决定）
- 参数配置：SN、曝光(ms)、增益、帧率均来自 appsettings.json
- 线程安全：_grabLock 互斥锁覆盖 Start/Stop 全流程
- 安全退出：StopGrabbing 通过 Task.Wait(timeoutMs) 等待线程真正退出
- 异常隔离：采集异常只记录不崩溃，进入下一帧重试

2.2 DetectionService (IDetectionService 实现)

生产者-消费者模式的检测引擎。内部使用 BlockingCollection<HObject>(容量=2) 作为帧队列，独立消费者线程执行 Process() 方法，完成形状匹配 + 针脚计数 + 合格判定。

- 队列容量为 2，满帧直接丢弃 (TryAdd 非阻塞)，保证相机线程不阻塞
- 最小匹配分阈值 0.65（低于此值视为未匹配到芯片）
- 状态机防重复：上升沿（新芯片）→ 高电平期（忽略重复帧）→ 下降沿（2.5s 超时复位）
- Process 内部所有 HObject 中间变量在 finally 中释放，防止内存泄漏
- ResultReady 事件发布检测结果，所有权转移给订阅方

2.3 TemplateService (ITemplateService 实现)

基于 Halcon 24.11 的 XLD 边缘子像素提取 + 形状模板创建服务。

- CreateTemplate: GenRectangle1 → EdgesSubPix → CreateShapeModelXld → GetShapeModelContours
- LoadTemplate: 从 TemplateConfig JSON + 参考图重建模板坐标与形状模板
- ExportConfig: 导出当前模板坐标（供 JSON 持久化，支持换产品切换）
- 资源所有权: TemplateRegion/ModelContours/ModelID 归本服务，ClearTemplate 是唯一释放入口

3. 数据访问层

Data 模块以 Prism IModule 方式加载，启动时自动 EnsureCreated 数据库。

- DbContext: AppDbContext (EF Core 8 + SQL Server)
- Repository: ProductInspectionRecordRepository / MemberRepository
- 统计服务: StatisticsService (Interlocked 计数 + 数据库聚合查询)
- 数据存储: 订阅 DetectionResultEvent (BackgroundThread)，异步写入数据库
- 实体 ProductInspectionRecord: 记录检测时间、产品型号、批次号、设备号、OK/NG 判定、针脚数

4. 基础设施层

Infrastruce 项目提供全局基础服务：

- ConfigService: 强类型配置中心，绑定 appsettings.json，支持热重载与 Changed 事件
- LogService: 日志服务，通过 LogPubSubEvent 广播到 SystemLogView
- HalconRuntime: Halcon 24.11 运行时初始化与卸载管理

5. 硬件控制层

Motion 模块封装所有硬件控制逻辑：

- MotionControlService: 雷赛 LTSMC 控制卡操作（连接、使能、启停、调速）
- SensorService: 读取传感器状态 (IN0 端口)，检测芯片是否到达

- SolenoidValueService: 电磁阀控制 (Open/Close) , 用于剔除不合格品
- 所有硬件参数 (IP、端口、速度、IO 位) 均从 appsettings.json 读取

III. 通信机制设计

1. Prism EventAggregator 事件总线

系统使用 Prism EventAggregator 作为跨模块通信的消息总线，实现模块间解耦。事件表如下：

事件名称	负载类型	发布者	订阅者	线程
ImageGrabbedEvent	ImageGrabbedPayload	CameraService	VisionWindowVM	UIThread
DetectionResultEvent	DetectionResult	VisionWindowVM	DataModule / DetectionRecordVM	Background / UI
LogPubSubEvent	LogEntry	LogService	SystemLogVM	UIThread
VisionStatusEvent	VisionStatus	VisionWindowVM	(状态栏)	UIThread
AppShutdownEvent	void	App.xaml	VisionModule / MotionModule	UIThread

2. ViewModel-to-View 事件

对于需要直接操作 UI 控件的场景（如 Halcon 窗口绘制），ViewModel 通过 C# 事件通知 View：

事件	类型	用途
ImageGrabbed	Action<HObject>	View 在 Halcon 窗口显示实时图像
DetectionResultUpdated	Action<DetectionResult>	View 绘制检测结果叠加图形
RequestTemplateCreate	Action	View 进入模板绘制模式
RequestCheckXld1Create	Action	View 进入检测区域 1 绘制模式
RequestCheckXld2Create	Action	View 进入检测区域 2 绘制模式
RequestDrawRoi	Action	View 创建 HDrawingObject 交互绘图
RequestClearRoi	Action	View 清除 HDrawingObject 并恢复显示

3. 生命周期管理

Prism 模块生命周期管理规则：

- VisionModule：OnInitialized 中订阅 AppShutdownEvent，在 OnShutdown 回调中依次释放 CameraService → TemplateService → DetectionService
- MotionModule：OnInitialized 中订阅 AppShutdownEvent，在 OnShutdown 回调中释放 MotionControlService
- DataModule：以 BackgroundThread 订阅 DetectionResultEvent，异步写入数据库
- IDestructible：DetectionRecordViewModel / SystemLogViewModel 实现 IDestructible 接口，在导航销毁时取消事件订阅

IV. 核心技术实现

1. 视觉检测算法

检测算法位于 `DetectionService.Process()` 方法，共 10 步：

- 步骤 1: `FindShapeModel` —— 形状模板匹配，获取芯片位姿 (`row/col/angle/score`)
- 步骤 2: 分数判断 —— `score < 0.65` 判定为 No found，直接返回
- 步骤 3: 状态机判断 —— 根据 `Stopwatch` 高精度时间戳判断是否为上升沿（新芯片）
- 步骤 4: `VectorAngleToRigid` —— 计算旋转平移矩阵，将模板变换到匹配位姿
- 步骤 5: `AffineTransContourXld` —— 生成匹配位姿下的模板轮廓（用于显示）
- 步骤 6: `GenRectangle2` —— 生成两个检测区域的旋转矩形（随芯片旋转）
- 步骤 7: `ReduceDomain` —— 在原图中提取两个检测区域的子图像
- 步骤 8: `CountPins` —— 对子图像依次执行：阈值分割 → 连通域分析 → 特征选择 → 计数
- 步骤 9: 合格判定 —— `pin1 ∈ [4,4]` 且 `pin2 ∈ [4,4]` 判定为 OK
- 步骤 10: 生成显示图 —— 叠加匹配轮廓 + 检测区域到原图

针脚计数 `CountPins` 内部流程：

```
//
H
//
H
//
H
//
H
```

2. 模板匹配算子清单

`TemplateService` 创建模板时使用的 Halcon 算子序列：

步骤	算子	用途
1	<code>GenRectangle1</code>	生成模板矩形 ROI
2	<code>ReduceDomain</code>	提取 ROI 子图像
3	<code>EdgesSubPix</code>	子像素边缘提取
4	<code>CreateShapeModelXld</code>	创建基于 XLD 轮廓的形状模板
5	<code>GetShapeModelContours</code>	获取模板轮廓（用于可视化）
6	<code>GenContourPolygonXld</code>	生成 ROI 边框 XLD 轮廓

3. 多线程架构

系统中存在 5 个并发线程，各线程职责明确：

线程	所属服务	职责	同步机制
UI 线程	WPF	界面渲染、用户交互	Dispatcher
采集线程	CameraService	<code>GrabImage</code> 循环	<code>_grabLock + volatile</code>
消费者线程	DetectionService	帧处理 + 检测	<code>_startStopLock + BlockingCollection</code>
日志线程	LogService	日志广播	EventAggregator

统计线程	StatisticViewModel	自动刷新	IPollTask
------	--------------------	------	-----------

线程安全治理措施：

- T1 — 跨线程状态标志使用 volatile（_isGrabbing、_running）
- T2 — 启停互斥锁（_grabLock、_startStopLock）覆盖 Start/Stop 全流程
- T3 — 后台线程退出通过 Task.Wait(timeoutMs) 等待确认，超时返回 false

4. 非托管资源管理

Halcon 的 HObject/HTuple 为非托管资源，必须显式释放。本系统采用以下规则：

- M1 — 异常隔离：单帧检测异常不传播，仅记录日志并返回 IsError=true
- M2 — 所有权明确：TemplateRegion/ModelContours/ModelID 归 TemplateService 所有，ClearTemplate 是唯一释放入口
- M3 — 所有权转移：每帧 Clone 后发布，接收方负责 Dispose；Process 返回的 DisplayImage 所有权归调用方
- M4 — 资源总量控制：BlockingCollection 容量=2，丢失帧不积累；_lastFrame 仅缓存最新一帧，旧帧先 Dispose 再替换

V. 已知风险与优化建议

1. 内存泄漏风险

编号	风险点	描述	治理措施	风险等级
M1	异常隔离	单帧检测异常不传播，仅记录后继续下一帧	已实现	低
M2	资源所有权	持有 ModelID/Contours，ClearTemp	已实现	低
M3	内存所有权转移	每帧由发布者，接收方负责 Dispose；返回 DisplayImage 所有权续继		中
M4	资源总量控制	缓存容量=2；_lastFrame 缓存旧帧先	已实现	低

2. 线程安全风险

编号	风险点	描述	治理措施	风险等级
T1	跨线程可见性	跨线程状态标志（_isGrabbing、_running）	volatile 关键字	低
T2	启停竞争	Start/Stop 同时调用导致状态专用互斥锁（_grabLock、_startStopLo		中
T3	后台线程退出	停止采集时后台线程仍访问已释放句柄	Task.Wait(timeout) 等待确认退出	高

3. 硬件耦合风险

编号	风险点	描述	治理措施	风险等级
H1	参数硬编码	机 SN、控制卡 IP、IO 端口等参数散落代码	配置化到 appsettings.json	低
H2	剔除延迟	剔除时与传送带线速度耦合，速度变化	ComputeRejectDelay() 根据当前速度实时	中
H3	速度突变	ChangeSpeed 需先停止传送带再调速	stop() + 调速 + Vmove() 三步流程	中

4. 数据库风险

编号	风险点	描述	治理措施	风险等级
D1	连接串安全	明文 sa/1 密码硬编码	配置化 + 可选 DPAPI 加密	高
D2	写入性能	检测主线程直接写库可能阻塞	BackgroundThread 异步写库，不阻塞主	中
D3	查询性能	长时间范围记录数达百万级	端侧聚合（按分钟 GROUP BY）+ 分页查询	中

5. 业务逻辑风险

编号	风险点	描述	治理措施	风险等级
L1	状态机可靠性	下降沿超时复位机制依赖 2.5s 定时	Stopwatch 高精度时间戳 + 可配置超时	中
L2	模板与相机耦合	创建模板时必须先打开相机	TemplateService 解耦，LoadTemplate 可离	低
L3	模板离线加载	换产品需重新绘制 ROI	TemplateConfig JSON 持久化坐标，支持一键加载	低
L4	漏帧处理	检测速度跟不上采集速度时丢帧	BlockingCollection 容量=2，TryAdd 非阻	中

6. 退出流程风险

编号	风险点	描述	治理措施	风险等级
X1	资源释放顺序	退出时采集线程仍在运行，访问已释放句柄	StopGrabbing 等待线程退出 + 超时机制	高

X2	模块卸载	模块卸载时未释放非托管资源 AppShutdownEvent 统一释放 + IDispose	中
----	------	---	---

7. 优化建议

- 建议 1: 引入 DPAPI 加密连接串，避免数据库密码明文存储
- 建议 2: 采集帧率与检测帧率解耦，加入动态帧率调节机制
- 建议 3: 增加看门狗定时器，监控相机连接状态与检测线程健康状态
- 建议 4: 模板版本管理，支持模板文件的历史版本回溯
- 建议 5: 增加日志文件持久化，支持按日期滚动归档
- 建议 6: 引入单元测试覆盖核心算法（CountPins、状态机逻辑）

VI. 文档修订历史

版本	日期	修订人	修订内容
v1.0	2026-09-04	系统自动生成	初始版本，覆盖系统架构、核心模块、通信机制、核心技术、风险与优化